LESSON 05

规则体系与 Spec 驱动开发

AI 与你之间的合同

Proposal | Specs | Design | Tasks | Implement



25-30 min · AI 与你之间的合同

AGENDA . 今日路线图

四件事，递进展开

01 vibe vs spec

没合同就只能赌运气

失败案例对比 . spec-driven 同需求

02 rules . guidelines . spec

三层叠加

三者对比表 . 协作图

03 OpenSpec 五段速览

5 个 markdown 文件

proposal . specs . design . tasks . implement

04 动手做 + 作业

走完整 5 段

add-dual-ma-strategy . 自测数字 . 提交清单

01

vibe vs spec

没合同就只能赌运气

一段对话，一个 lookahead bias 翻车

对话记录（vibe 模式）

我： 帮我写个 MA 策略，金叉买入

Claude: 好的，代码如下——

```
sig = (ma_short > ma_long).astype(int)
```

我： 回测一下

Claude: sharpe=3.8, cum=0.42

我： 真棒！直接上生产吧

[三个月后] 实盘: cum=-0.15 ???

为什么翻车?

Lookahead Bias

sig 用了当天 close 计算的均线，
回测读取了未来数据，虚高收益。

无接口约定

没有定义返回值是 {0,1} 还是 {-1,0,1}，
仓位计算逻辑全靠 AI 猜。

无边界条件

均线窗口期内的 NaN 怎么处理?
没有 spec， AI 每次处理方式不同。

没有 spec = 没有合同 = 回测结果依赖 AI 的心情，上生产必翻

01 . spec-driven 同需求

先 spec 再 implement，结果截然不同

Vibe Coding

- × 直接说需求，AI 直接写代码
- × 接口由 AI 自由发挥
- × 回测数字虚高（lookahead）
- × 换一次 AI 行为不一致
- × 上生产才发现问题
- × 调试成本：极高

VS

Spec-Driven

- ✓ 先写 proposal + specs.md
- ✓ 接口约定在先，AI 实现在后
- ✓ 防 lookahead 写进 specs
- ✓ specs 可复核，行为可重现
- ✓ CI 验证，上线前已知结果
- ✓ 调试成本：极低

Spec 不是额外负担——它是让 AI 产出可信结果的唯一方式

02

rules . guidelines . spec

三层叠加

02 . 三者对比

适用范围 . 强制度 . 维护成本 . 例子

	Rules	Guidelines	Spec
适用范围	整个仓库 / 全局	文件类型 / 模块	单个 feature / change
强制度	AI 必须遵守	AI 默认遵守，可显式覆盖	AI 按约定实现，可审查
维护成本	低——改动少	中——随风格演进	高——每个 feature 都要写
例子	UTC / 注释用中文	命名风格 / 日志格式	dual_ma 函数签名

02 . 三者协作图

rules 全局 . guidelines 风格 . spec 单 feature

RULES（CLAUDE.md / .cursor/rules/）

全局约束： UTC / 中文注释 / 命名规范 / 安全底线

GUIDELINES（.cursor/rules/*.mdc）

风格约定：日志格式 / 返回值类型 / 错误处理 / 测试命名

SPEC（openspec/changes/<feature>/）

单 feature 合同： proposal + specs + design + tasks + implement

例： add-dual-ma-strategy — 函数签名、金叉死叉规则、防 lookahead 约定

三层不冲突： rules 是底线， guidelines 是默认风格， spec 是单次约定

03

OpenSpec 五段速览

5 个 markdown 文件

Setup （可选）

```
npm install -g openspec  -  cd lesson5/project && openspec init
```

CLI 是可选便利层——纯手写 5 段 markdown 也行；本课样本两种方式产物一致。

03 . OpenSpec 五段总览

proposal - specs - design - tasks - implement

Proposal

`proposal.md`

提出需求

动机 + 痛点

>

Specs

`specs.md`

接口约定

函数签名 + 规则

>

Design

`design.md`

技术决策

方案对比 + 选择

>

Tasks

`tasks.md`

开发清单

原子任务列表

>

Implement

（代码文件）

代码实现

AI 按 spec 写

openspec/changes/<feature-name>/ - 5 个文件，完整记录一个 feature 的生命周期

03 . 各段角色一句话

一句话定义 + 一个反例

Proposal	Specs	Design	Tasks	Implement
提出「为什么做」和「做什么」	定义「做到什么程度算完成」	记录「为什么这样设计」	拆成「可独立完成的原子步骤」	代码与 specs 对齐，可验证
反例： 直接写实现，没有说清楚为什么需要这个 feature	反例： specs 写成自然语言故事，无法验证是否实现	反例： 只记录最终方案，丢失备选方案和决策理由	反例： task 写成「实现双均线策略」，一句话包含 5 个操作	反例： 实现完后 specs.md 没有更新，两者不一致

04

动手做： add-dual-ma-strategy

走完整 5 段

04 . proposal.md

5 行核心——为什么做 + 做什么

openspec/changes/add-dual-ma-strategy/proposal.md （节选）

```
## 问题
lesson5/project/strategies/ 只有 buy_and_hold（基线）和 ma_crossover_7
（单均线突破），缺少反转信号——无法教学多空切换。

## 提案
实现双均线交叉策略 dual_ma：短均线（默认 7 日）上穿长均线
（默认 14 日） -> 信号 +1（做多）；短均线下穿 -> 信号 0（空仓）；
其他情况维持上一日信号。
```

动机清晰 说明现有策略局限，不是「想做就做」

范围明确 7 日 /14 日 /+1/0，不含 -1 （简化教学）

无实现细节 proposal 只描述「做什么」，不提「怎么做」

04 . specs.md 接口段

函数签名 + 金叉 / 死叉 / 其他 信号规则表

函数签名

```
dual_ma(df: pd.DataFrame,
        short_window: int = 7,
        long_window:  int = 14) -> pd.Series
```

防 lookahead 约定

```
df["close"].shift(1).rolling(N).mean()
```

确保信号决定时不读今日 close

兼容性：可被 backtest.run_backtest(dual_ma) 直接调用

行为规则表

状态	条件	当日信号
金叉	ma_s > ma_l 且昨日 ma_s <= ma_l	+1
死叉	ma_s < ma_l 且昨日 ma_s >= ma_l	0
其他	无交叉（持仓维持）	同昨日信号

04 . design.md 备选方案

做决策前先列方案，记录为什么选 B

方案	描述	实现复杂度	教学清晰度	选择
A	close vs MA（简单突破）	低	高，但和 ma_crossover_7 雷同	X
B	短 MA vs 长 MA（金叉死叉）	中	高，经典量化教科书例	O
C	MACD（DIF/DEA）	高	高，但引入 EMA 超出 25 min	X

选 B：中等复杂度 + 经典教学例子，比 A 多反转信号，比 C 少额外概念

两层防御 lookahead — 代码骨架

design.md — 状态保持骨架（5 行 Python 防 lookahead 双层）

```
cross_up = (ma_s > ma_l) & (ma_s_prev <= ma_l_prev)
cross_dn = (ma_s < ma_l) & (ma_s_prev >= ma_l_prev)
sig = pd.Series(np.nan, index=df.index)
sig[cross_up] = 1
sig[cross_dn] = 0

sig = sig.ffill().fillna(0)  # 持仓维持 + 起始空仓
```

第一层 策略层： df["close"].shift(1).rolling(N).mean() — 均线只用昨日及更早数据

第二层 backtest 层： pos = sig.shift(1) — 今天信号用于明天仓位，不读今天收盘

6 项原子任务——动作动词开头

openspec/changes/add-dual-ma-strategy/tasks.md

---

[x] 1. 在 **lesson5/project/strategies/__init__.py** 导出 dual_ma

[x] 2. 写 **lesson5/project/strategies/dual_ma.py**：函数签名、design 骨架、中文注释

[x] 3. 跑自测：**python strategies/dual_ma.py** 无 traceback

[x] 4. 把 specs.md 接口段合并到 **lesson5/project/openspec/specs/strategies.md**

[x] 5. 写 **IMPLEMENTATION.md**，记录实施日期、数字、与 design 偏差

[x] 6. 提交 **commit**

每条 task = 一个动作动词 + 一个目标 + 可验证结果　AI 完成后可勾选

04 . dual_ma.py 关键代码

10 行核心逻辑——严格按 specs.md 约定实现

lesson5/project/strategies/dual_ma.py （核心片段 10 行）

```
ma_short = df["close"].shift(1).rolling(short_window).mean()
ma_long  = df["close"].shift(1).rolling(long_window).mean()
ma_short_prev = ma_short.shift(1)
ma_long_prev  = ma_long.shift(1)
cross_up = (ma_short > ma_long) & (ma_short_prev <= ma_long_prev)
cross_dn = (ma_short < ma_long) & (ma_short_prev >= ma_long_prev)
sig = pd.Series(np.nan, index=df.index)
sig[cross_up] = 1.0
sig[cross_dn] = 0.0

sig = sig.ffill().fillna(0).astype(int)
```

shift(1) 两次：策略层（均线计算） + backtest 层（仓位延迟） = 两层防 lookahead

90 日 BTCUSDT 1d — 真实数字（2026-05-05）

执行命令（lesson5/project/ 目录下）

```
python -c "from strategies import dual_ma; from backtest import run_backtest; print(run_backtest(dual_ma, limit=90)['summary'])"
```

输出：

```
sharpe=1.1563  max_dd=-0.0938  win_rate=0.5870  cum=0.0911
```

中文总结（run_backtest 生成）

BTCUSDT 在最近 90 根 K 线上的回测结果：累计上涨 9.11%，年化夏普 1.16，最大回撤 -9.38%，信号触发日胜率 58.70%。夏普 > 1 说明风险调整后收益尚可，配合曲线图可看到回撤分布，建议结合更长历史数据复跑确认稳健性。

spec 写了防 lookahead -> 数字可信； spec 写了兼容 run_backtest -> 0 行胶水代码

从「暂无」到「已实现 dual_ma 」— Before / After

Before（specs/strategies.md）

```
## 策略接口

暂无已正式 spec 的策略。
```

策略目录里已有 buy_and_hold 和 ma_crossover_7，
但没有正式接口约定——任何人都可以随意修改

返回值格式，没有依据。

After（specs/strategies.md）

```
## 已实现策略

### dual_ma
dual_ma(df, short=7, long=14)
  -> pd.Series  # 值：{0, 1}
金叉 +1 / 死叉 0 / 维持昨日

防 lookahead: shift(1).rolling
```

specs.md 是活文档——每次 implement 结束都要回来补充，让它成为可查的合同

关键认知

spec = 合同

AI 与你之间的书面约定，可以验证、可以审计

rules = 底线

不管做什么 feature，这些约定永远成立

guidelines = 味道

默认风格，AI 遵循，你可以用 prompt 覆盖



这三件事会让 OpenSpec 失效

AP1

200 行 spec 流水账

把每行代码的意图都写进 specs.md ，
变成实现文档。

改： specs 只写接口约定（函数签名、
输入输出、边界条件），实现细节
留给 design.md 和代码注释。

AP2

写了 specs/ 但不维护

实现完成后 specs.md 不更新，
导致文档与代码不一致。

改：每次 implement 任务完成后，
回来更新 specs/strategies.md ，
把「暂无」改为已验证接口。

AP3

把 spec 当事后审计

feature 做完再补 specs.md ，
失去「约定在先」的价值。

改： spec 先写， implement 后做；
顺序是： proposal -> specs ->
design -> tasks -> implement 。

05

作业 + 收束

30 分钟，写 5 段

5 候选方向 +4 条评分维度

A

RSI 策略

提案： RSI < 30 买入
RSI > 70 卖出
走完 5 段，含防

lookahead 约定

B

止损规则

提案：日内亏损超 2%
强制平仓

设计状态保持骨架

C

多品种对比

提案：同一策略跑
BTC + ETH

设计多资产接口

D

自选策略

提案你熟悉的量化
信号
写出可验证的

specs.md

E

非量化场景

提案代码审查 /
日志分析等 feature

OpenSpec 5 段全走

评分： 1 specs 可验证 2 防 lookahead 明确 3 tasks 原子化 4 specs.md 与代码对齐

05 . 作业提交清单

4-5 个 markdown + 3 句话回顾 + 加分项

必交（4-5 个 markdown）

proposal.md feature 动机 + 描述（5-10 行）

specs.md 函数签名 + 行为规则表（可验证）

design.md 至少 2 个备选方案 + 选择理由

tasks.md 6 条原子任务（动作动词开头）

（可选）代码 实现你选的策略或 feature

3 句话回顾 + 加分项

句 1 这个 feature 的动机是什么？

句 2 specs 里最难写的是哪条规则？为什么？

句 3 如果重来，你会在哪段多花时间？

加分项

提交 IMPLEMENTATION.md：记录实施日期、
真实回测数字、与 design 的偏差说明

提交方式：openspec/changes/<your-feature>/ 目录下 + 3 句话说明，发到课程群即可

学完这课你能做什么

4 条 Takeaway

1 用 OpenSpec 5 段管理 feature

不再靠记忆和口头约定，每个 feature 都有 proposal + specs + design + tasks + 代码全套文档。

2 区分 rules / guidelines / spec

知道什么应该写进 CLAUDE.md （全局底线），什么放 guidelines （风格），什么放 specs （单次约定）。

3 写出可验证的 specs

specs.md 里的约定可以用测试或 backtest 验证，不再是「AI 大概应该这样做」的模糊说法。

4 防 lookahead 双层约定

策略层 shift(1).rolling + backtest 层 pos.shift(1)，两层防御让回测数字可信，可直接用于复盘。

下节课（Lesson 6）：spec 说「做什么」，harness 说「如何持续验证」——CI + 回归 + 测试驱动

预告 . Lesson 6

spec = What ； harness = How to keep going

Lesson 6 核心主题

OpenSpec 解决了「做什么、做到什么程度」；但每次新 commit 进来，怎么保证之前的 spec 还成立？答案是 harness （测试驱动回归）：

用测试把 specs 里的约定固化成可执行检查， CI 自动回归。

Spec （Lesson 5， 你已学会）

. spec 定义：接口约定 + 边界条件

. spec 存在： markdown （人可读）

. spec 验证：人工 review + backtest

. spec 时效：每次 feature 写一次

>

Harness （Lesson 6， 下节课）

. harness 定义：可执行的 spec 映射

. harness 存在： pytest / CI pipeline

. harness 验证：自动化，每次 commit

. harness 时效：持续守护，一直运行

Have fun ． 延伸阅读

Spec 先行——让 AI 造轮子之前先签合同

OpenSpec 规范（本课使用）

`github.com/anthropics/openspec`

OpenSpec 官方仓库，含 5 段模板和示例 feature，本课 openspec/ 目录即照此搭建

课堂 change： add-dual-ma-strategy

`lesson5/project/openspec/changes/add-dual-ma-strategy/`

本节课走完的完整 5 段文件： proposal / specs / design / tasks / dual_ma.py

SDD 博客： Spec-Driven Development

`martinfowler.com/articles/spec-driven-dev.html`

Martin Fowler 论 Spec 先行的工程价值，与 TDD 的异同，经典必读

Cursor Rules 文档

`docs.cursor.com/context/rules-for-ai`

rules / guidelines 在 Cursor 里的配置方式，对应本课三层体系中的前两层